Question-3

Runxin Shen

2026-10-29

Modification 1: Training-Only Feature Selection

Setup

```
library(tidyverse)
library(infer)
library(randomForest)
library(tidymodels)
library(yardstick)
library(here)

## 1. LOAD RAW DATA

var_names <- read_csv(
  here("data", "biomarker-raw.csv"),
  col_names = FALSE,
  n_max = 2,
  col_select = -(1:2)
) %>%
  t() %>%
  as_tibble() %>%
  rename(name = V1,
         abbreviation = V2) %>%
  na.omit()

biomarker_raw <- read_csv(
  here("data", "biomarker-raw.csv"),
  skip = 2,
  col_select = -2L,
  col_names = c('group',
                'empty',
                pull(var_names, abbreviation),
                'ados'),
  na = c('-', '')
) %>%
  filter(!is.na(group)) %>%
  select(group, ados, everything())
```

```
## 2. TRAIN / TEST SPLIT
```

```r
# Split before preprocessing and feature selection
set.seed(101422)

biomarker_split <- biomarker_raw %>%
  initial_split(prop = 0.8, strata = group)

train_raw <- training(biomarker_split)
test_raw <- testing(biomarker_split)

# Check group sizes
table(train_raw$group)
```

```
##
## ASD  TD
##  60  62
```

```r
table(test_raw$group)
```

```
##
## ASD  TD
##  16  16
```

```r
## 3. PREPROCESSING

# Trim outliers at +/- 3
trim <- function(x, .at = 3) {
  x[abs(x) > .at] <- sign(x[abs(x) > .at]) * .at
  return(x)
}

# Separate protein measurements
train_proteins <- train_raw %>%
  select(-group, -ados) %>%
  as.matrix()

test_proteins <- test_raw %>%
  select(-group, -ados) %>%
  as.matrix()

# Log transformation
train_log <- log10(train_proteins)
test_log <- log10(test_proteins)

# Center and scale using training data only
train_scaled <- scale(train_log)

train_center <- attr(train_scaled, "scaled:center")
train_scale <- attr(train_scaled, "scaled:scale")

test_scaled <- scale(
  test_log,
  center = train_center,
```

```
  scale = train_scale
)

# Trim outliers
train_scaled <- trim(train_scaled)
test_scaled <- trim(test_scaled)

# Reconstruct clean datasets
train_clean <- bind_cols(
  train_raw %>% select(group, ados),
  as_tibble(train_scaled)
)

test_clean <- bind_cols(
  test_raw %>% select(group, ados),
  as_tibble(test_scaled)
)


## 4. MULTIPLE TESTING

# Function to compute t-tests
test_fn <- function(.df) {
  t_test(
    .df,
    formula = level ~ group,
    order = c('ASD', 'TD'),
    alternative = 'two-sided',
    var.equal = FALSE
  )
}

# Perform all tests using training data only
ttests_out <- train_clean %>%
  select(-ados) %>%
  pivot_longer(
    -group,
    names_to = 'protein',
    values_to = 'level'
  ) %>%
  nest(data = c(level, group)) %>%
  mutate(ttest = map(data, test_fn)) %>%
  unnest(ttest) %>%
  arrange(p_value) %>%
  mutate(
    m = n(),
    hm = log(m) + 1/(2*m) - digamma(1),
    rank = row_number(),
    p.adj = m * hm * p_value / rank
  )

# Select top 10 proteins
proteins_s1 <- ttests_out %>%
```

```
  slice_min(p.adj, n = 10, with_ties = FALSE) %>%
  pull(protein)



## 5. RANDOM FOREST

# Training predictors and response
predictors <- train_clean %>%
  select(-group, -ados)

response <- train_clean %>%
  pull(group) %>%
  factor()

# Fit random forest using training data only
set.seed(101422)

rf_out <- randomForest(
  x = predictors,
  y = response,
  ntree = 1000,
  importance = TRUE
)

# Check training OOB errors
rf_out$confusion
```

```
##     ASD TD class.error
## ASD  41 19   0.3166667
## TD   12 50   0.1935484
```

```
# Select top 10 proteins
proteins_s2 <- rf_out$importance %>%
  as_tibble() %>%
  mutate(protein = rownames(rf_out$importance)) %>%
  slice_max(MeanDecreaseGini, n = 10,
            with_ties = FALSE) %>%
  pull(protein)



## 6. HARD INTERSECTION

# Proteins selected by both methods
proteins_sstar <- intersect(
  proteins_s1,
  proteins_s2
)

print(proteins_sstar)
```

```
## [1] "DERM"        "Calcineurin" "MAPK2"       "FSTL1"       "IgD"
```

```
## [6] "MAPK14"
```

```
length(proteins_sstar)
```

```
## [1] 6
```

```
selection_summary <- tibble(
  Method = c(
    "Multiple Testing",
    "Random Forest",
    "Hard Intersection"
  ),
  Number_of_Proteins = c(
    length(proteins_s1),
    length(proteins_s2),
    length(proteins_sstar)
  )
)
```

```
## 7. LOGISTIC REGRESSION

# Prepare training data
train_model <- train_clean %>%
  select(group, any_of(proteins_sstar)) %>%
  mutate(class = factor(group == 'ASD',
                        levels = c(FALSE, TRUE))) %>%
  select(-group)

# Prepare testing data
test_model <- test_clean %>%
  select(group, any_of(proteins_sstar)) %>%
  mutate(class = factor(group == 'ASD',
                        levels = c(FALSE, TRUE))) %>%
  select(-group)

# Fit model on training set only
fit <- glm(
  class ~ .,
  data = train_model,
  family = 'binomial'
)


## 8. FINAL EVALUATION

# Predict on the testing set
test_predictions <- test_model %>%
  mutate(
    pred = predict(fit, newdata = test_model,
                   type = 'response'),
    predicted_class = factor(
      pred > 0.5,
      levels = c(FALSE, TRUE)
```

```
    )
  )

# Classification metrics
test_metrics <- bind_rows(
  accuracy(test_predictions,
           truth = class,
           estimate = predicted_class),

  sens(test_predictions,
       truth = class,
       estimate = predicted_class,
       event_level = 'second'),

  spec(test_predictions,
       truth = class,
       estimate = predicted_class,
       event_level = 'second'),

  roc_auc(test_predictions,
          truth = class,
          pred,
          event_level = 'second')
)

print(test_metrics)
```

```
## # A tibble: 4 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.656
## 2 sens     binary         0.5
## 3 spec     binary         0.812
## 4 roc_auc  binary         0.723
```

I divided the dataset into 122 training subjects and 32 testing subjects using stratified sampling. Preprocessing parameters and feature selection were determined using only the training data to avoid data leakage. Multiple testing and random forest each selected 10 proteins, with six proteins appearing in both sets: DERM, Calcineurin, MAPK2, FSTL1, IgD, and MAPK14.

The logistic regression model achieved a testing accuracy of 65.6%, sensitivity of 50.0%, specificity of 81.2%, and AUROC of 0.723. This procedure provides a more appropriate evaluation of out-of-sample performance. However, without evaluating the original procedure on the same test subjects, I cannot determine the exact effect of this modification on classification accuracy.

Modification 2: Increasing the Number of Selected Proteins

I increased the number of top predictive proteins selected by each method from 10 to 20 while keeping the hard intersection rule. This modification allows more candidate proteins to enter the final classification model.

```
## 9. MODIFICATION 2: TOP 20

# Top 20 from multiple testing
proteins_s1_20 <- ttests_out %>%
  slice_min(p.adj, n = 20, with_ties = FALSE) %>%
  pull(protein)

# Top 20 from random forest
proteins_s2_20 <- rf_out$importance %>%
  as_tibble() %>%
  mutate(protein = rownames(rf_out$importance)) %>%
  slice_max(MeanDecreaseGini, n = 20,
            with_ties = FALSE) %>%
  pull(protein)

# Hard intersection of Top 20
proteins_top20 <- intersect(
  proteins_s1_20,
  proteins_s2_20
)

print(proteins_top20)
```

```
##  [1] "DERM"            "Calcineurin"     "MAPK2"           "FSTL1"
##  [5] "CXCL16, soluble" "RELT"            "IgD"             "MAPK14"
##  [9] "Notch 1"         "TSP4"            "DKK3"
```

```
length(proteins_top20)
```

```
## [1] 11
```

Modification 3: Fuzzy Intersection

Instead of requiring proteins to appear in both Top 10 lists, I used a rank-based fuzzy intersection. Each protein received a membership score based on its ranking under multiple testing and random forest. The minimum of the two membership scores was used as the fuzzy intersection score, and the 10 highest-scoring proteins were selected.

```
## 10. MODIFICATION 3: FUZZY INTERSECTION

# Rank proteins by multiple testing
ttest_ranks <- ttests_out %>%
  arrange(p.adj) %>%
  transmute(
    protein,
    rank_ttest = row_number()
  )

# Rank proteins by random forest importance
rf_ranks <- rf_out$importance %>%
  as_tibble() %>%
```

```
  mutate(protein = rownames(rf_out$importance)) %>%
  arrange(desc(MeanDecreaseGini)) %>%
  transmute(
    protein,
    rank_rf = row_number()
  )

# Define fuzzy membership scores
fuzzy_results <- inner_join(
  ttest_ranks,
  rf_ranks,
  by = "protein"
) %>%
  mutate(
    membership_ttest = exp(-(rank_ttest - 1) / 10),
    membership_rf = exp(-(rank_rf - 1) / 10),
    fuzzy_score = pmin(
      membership_ttest,
      membership_rf
    )
  ) %>%
  arrange(desc(fuzzy_score),
          rank_ttest + rank_rf)

# Select top 10 by fuzzy intersection score
proteins_fuzzy <- fuzzy_results %>%
  slice_head(n = 10) %>%
  pull(protein)

print(proteins_fuzzy)
```

```
##  [1] "DERM"            "MAPK2"           "Calcineurin"     "FSTL1"
##  [5] "IgD"             "MAPK14"          "Notch 1"         "CXCL16, soluble"
##  [9] "TSP4"            "DKK3"
```

```
length(proteins_fuzzy)
```

```
## [1] 10
```

```
## 11. COMPARE EXPERIMENTS

evaluate_panel <- function(proteins, experiment) {

  train <- train_clean %>%
    select(group, all_of(proteins)) %>%
    mutate(class = factor(group == "ASD",
                          levels = c(FALSE, TRUE))) %>%
    select(-group)

  test <- test_clean %>%
    select(group, all_of(proteins)) %>%
    mutate(class = factor(group == "ASD",
```

```
                            levels = c(FALSE, TRUE))) %>%
    select(-group)

  model <- glm(
    class ~ .,
    data = train,
    family = "binomial"
  )

  pred <- predict(model, newdata = test,
                  type = "response")

  predicted_class <- factor(
    pred > 0.5,
    levels = c(FALSE, TRUE)
  )

  tibble(
    Experiment = experiment,
    Proteins = length(proteins),
    Accuracy = accuracy_vec(test$class, predicted_class),
    Sensitivity = sens_vec(
      test$class, predicted_class,
      event_level = "second"
    ),
    Specificity = spec_vec(
      test$class, predicted_class,
      event_level = "second"
    ),
    AUROC = roc_auc_vec(
      test$class, pred,
      event_level = "second"
    )
  )
}

comparison <- bind_rows(
  evaluate_panel(proteins_sstar, "Top 10 - Hard"),
  evaluate_panel(proteins_top20, "Top 20 - Hard"),
  evaluate_panel(proteins_fuzzy, "Top 10 - Fuzzy")
)

knitr::kable(
  comparison,
  digits = 3,
  caption = "Comparison of Protein Selection Methods",
  align = "lccccc"
)
```

Table 1: Comparison of Protein Selection Methods

Experiment	Proteins	Accuracy	Sensitivity	Specificity	AUROC
Top 10 - Hard	6	0.656	0.500	0.812	0.723
Top 20 - Hard	11	0.625	0.562	0.688	0.699
Top 10 - Fuzzy	10	0.625	0.562	0.688	0.699

Results and Discussion

Modification 1: Training-Only Selection

Feature selection was performed using only training data to avoid data leakage. The resulting six-protein model achieved 65.6% accuracy and an AUROC of 0.723. A direct comparison with the original analysis is still needed.

Modification 2: Top 20 Proteins

Increasing the number of selected proteins resulted in an 11-protein panel. Accuracy decreased from 65.6% to 62.5%, and AUROC decreased from 0.723 to 0.699, suggesting that adding more proteins did not improve overall performance.

Modification 3: Fuzzy Intersection

The fuzzy intersection selected 10 proteins and achieved the same accuracy (62.5%) and AUROC (0.699) as the Top 20 hard intersection, using one fewer protein.

Conclusion

The Top 10 hard intersection achieved the best overall performance. However, results are based on a single train-test split, so further validation is needed.